
CityPulse

Intelligent Parking & Traffic Control Platform

System Architecture Document

Contents

1	Architectural Style	3
1.1	NestJS Backend Module Map	3
2	System Layers	4
3	Service Boundaries	4
4	Data Flow Diagrams	4
4.1	Flow A — Real-Time Slot Availability	5
4.2	Flow B — Owner Registers a Parking Space	6
4.3	Flow C — Driver Finds Nearby Parking	6
4.4	Flow D — Booking & Payment (future planned)	7
5	Messaging & Event Architecture	7
6	Security Architecture — Permission-Based RBAC	8
6.1	Authentication	8
6.2	PDPO 2025 Compliance Readiness	8
6.3	Role Definitions	8
6.4	Permission Catalogue	9
6.5	Role to Permission Mapping	9
6.6	Guard Chain	10
6.7	JWT Token Payload	10
6.8	RBAC Entity Relationships	10
7	Database Design	10
7.1	PostgreSQL Core Schema	11
7.2	MongoDB Collections	11
7.3	Redis Key Patterns	11
7.4	PostGIS Nearby Search Query	11
8	Deployment Architecture	11
8.1	Service Containers	12
8.2	Environment Configuration	12
9	Design Patterns Applied	13

1. Architectural Style

CityPulse uses a **layered architecture** with an **event-driven core**. The NestJS backend is a **modular monolith** — clean module boundaries extractable to separate services in future planned via the Strangler Fig pattern. FastAPI is the only hard service split, forced by the Python language boundary for AI/ML workloads.

Why not a monolith

The AI/ML workload — RTSP ingestion, OpenCV, PyTorch/ONNX inference — is Python-native and cannot be embedded into a Node.js process. The separation is forced by language boundary: FastAPI handles AI, NestJS handles business logic.

Why not full microservices

The backend does not split into many services because this is a three-person team building a research prototype. Over-splitting introduces operational overhead with no proportional benefit at this scale. Module boundaries are kept clean so any module can be extracted as a separate service later without rewriting.

1.1 NestJS Backend Module Map

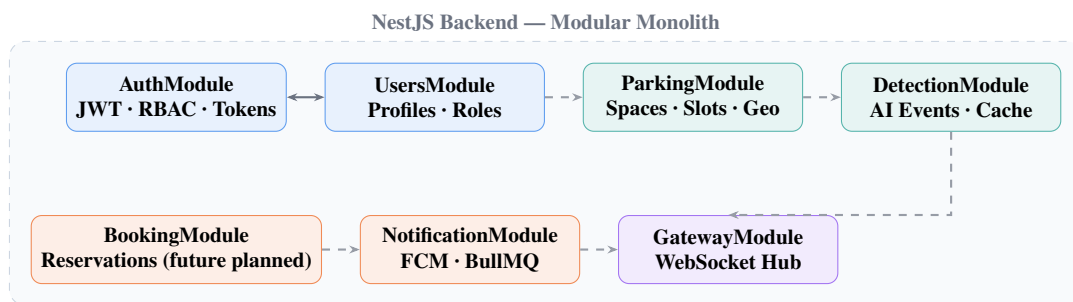


Figure 1: NestJS modular monolith — internal module structure.

2. System Layers

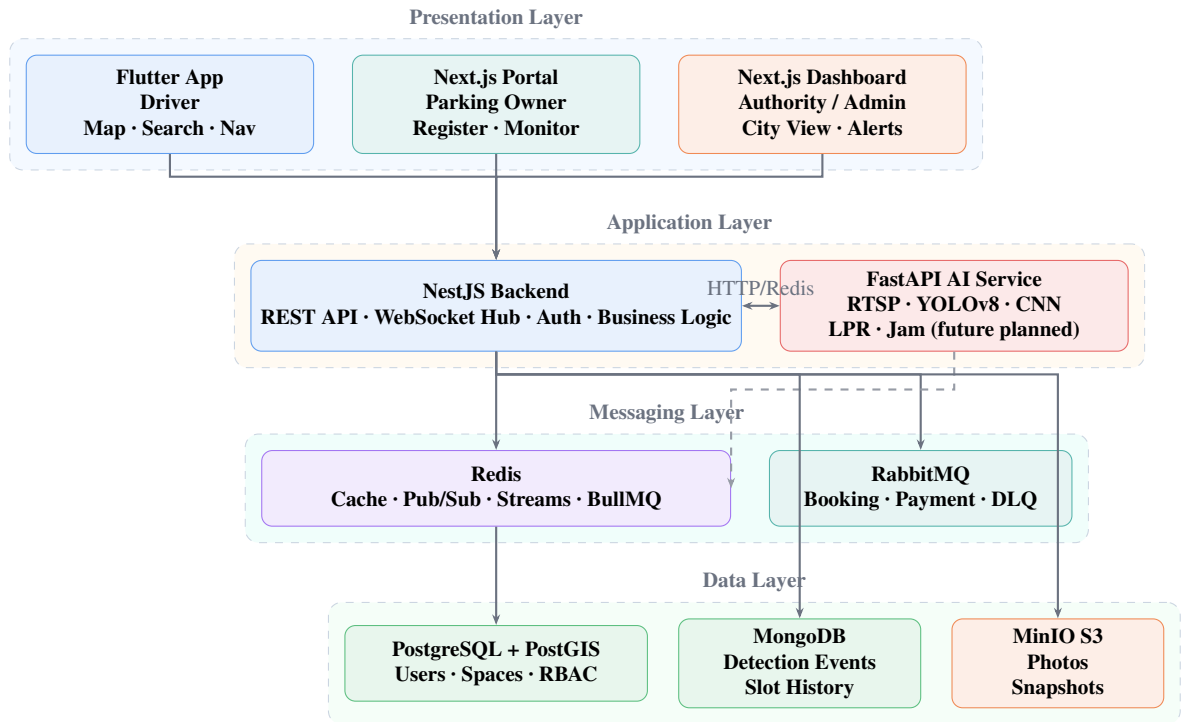


Figure 2: Full CityPulse system layers.

3. Service Boundaries

Service	Owns	Communicates via
NestJS Backend	Auth, business rules, REST API, Web-Socket hub, geospatial queries, job orchestration	HTTPS (clients), Redis Pub/Sub subscriber, RabbitMQ, internal HTTP caller
FastAPI AI Service	RTSP ingestion, frame extraction, all ML inference, slot status events	Redis Pub/Sub publisher, Redis Streams publisher, internal HTTP responder

Table 1: Service ownership and communication contracts.

Boundary rule: NestJS never touches a camera stream. FastAPI never touches client auth or business rules. They communicate through two channels: **Redis Pub/Sub** for async slot status events (FastAPI publishes, NestJS subscribes), and **internal HTTP** for synchronous calls such as registering a new camera or triggering on-demand inference.

4. Data Flow Diagrams

4.1 Flow A — Real-Time Slot Availability

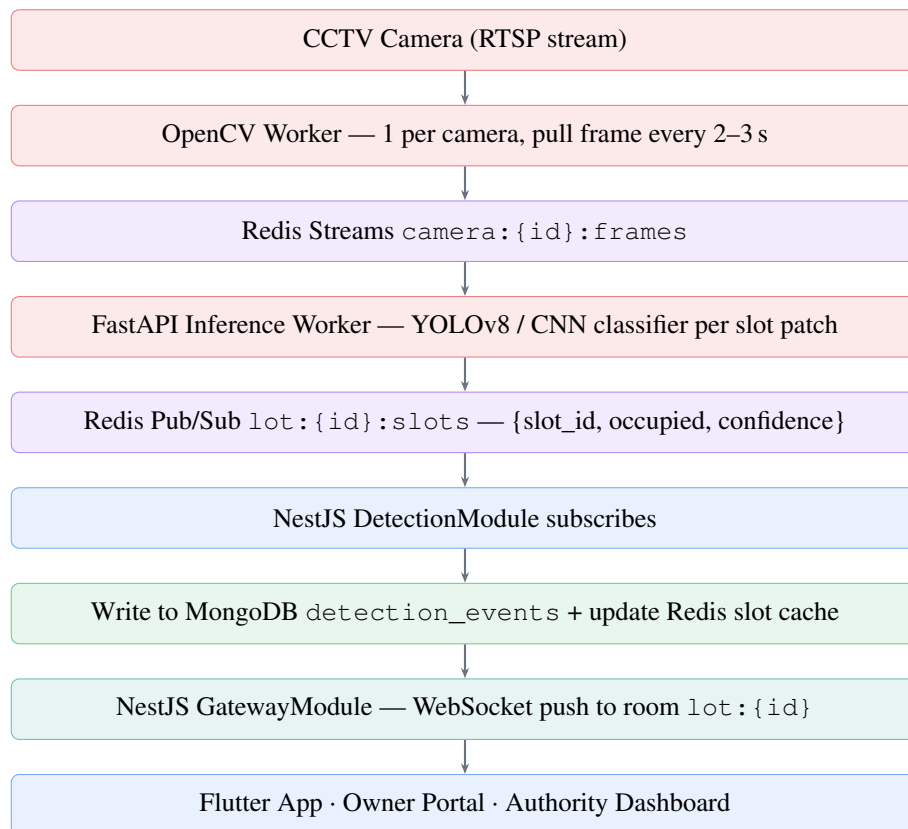


Figure 3: Flow A — CCTV frame to live client update.

4.2 Flow B — Owner Registers a Parking Space

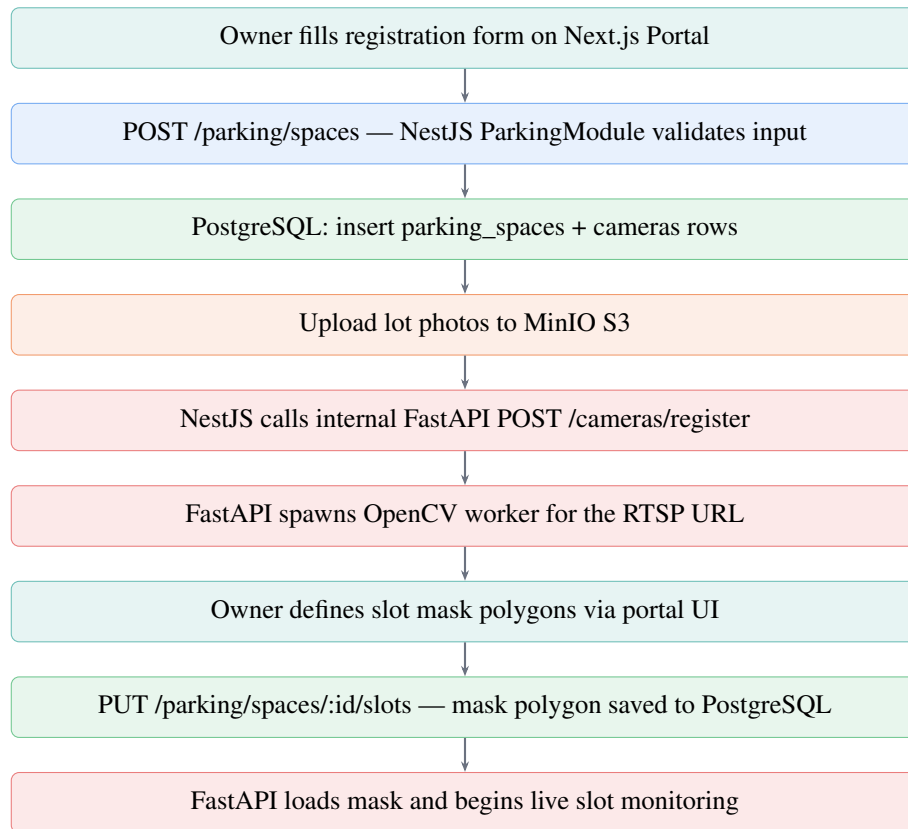


Figure 4: Flow B — Parking space registration and camera onboarding.

4.3 Flow C — Driver Finds Nearby Parking

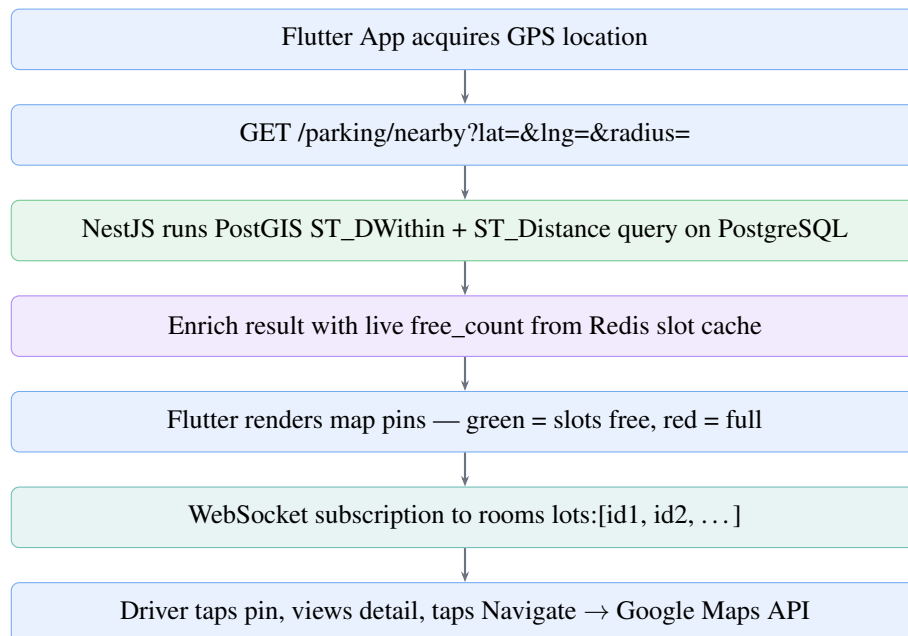


Figure 5: Flow C — Driver nearby parking search with live updates.

4.4 Flow D — Booking & Payment (future planned)

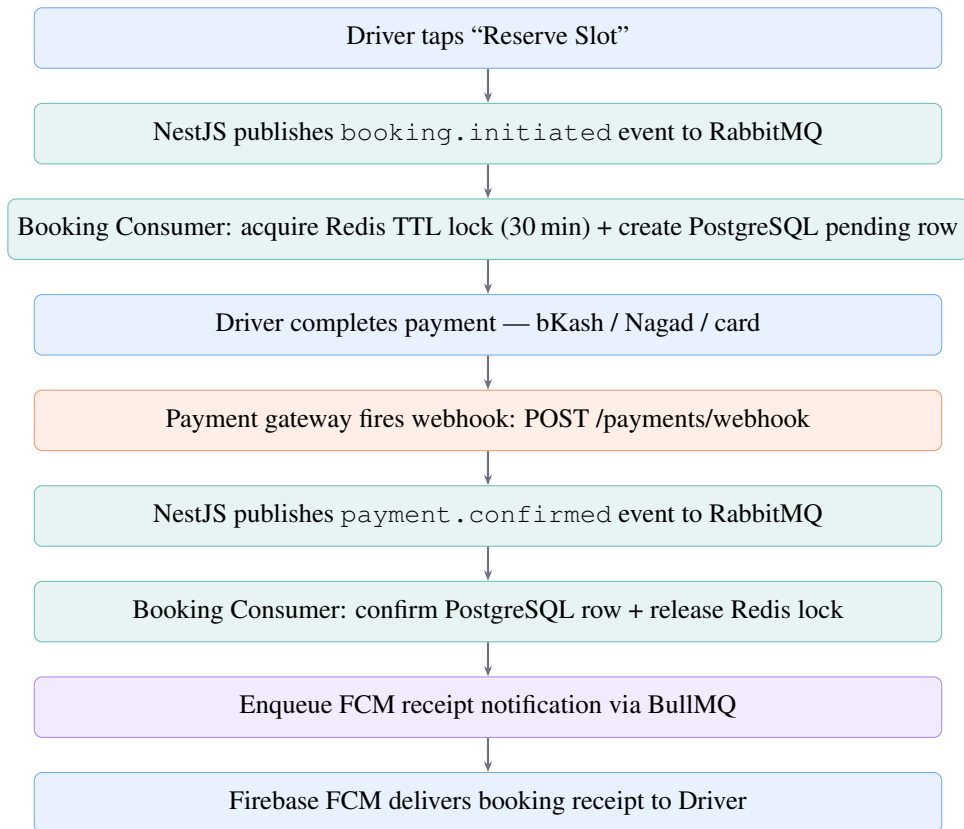


Figure 6: Flow D — Booking and payment with guaranteed RabbitMQ delivery (future planned). If consumer is down, message is held in queue and retried on recovery.

5. Messaging & Event Architecture

Use Case	Tool	Guarantee	Reason
Slot state cache	Redis key-value	Best effort	Sub-ms in-memory reads
Live slot updates	Redis Pub/Sub	Fire-and-forget	Next cycle corrects any missed update
CCTV frame pipeline	Redis Streams	At-least-once	Consumer groups, replay on failure
FCM / background jobs	BullMQ on Redis	At-least-once	Retry, delay, cron built in
Booking + payment	RabbitMQ	Guaranteed DLQ	+ Cannot lose a payment event

Table 2: Messaging tool selection by use case.

Why this split is intentional: Slot updates are stateless — if a client misses one push, the next inference cycle corrects it in 2–3 seconds. Losing a slot update is acceptable. Losing a payment event is not. These two requirements demand different tools. Using RabbitMQ for everything would be over-engineering; using Redis Pub/Sub for payments would be unsafe.

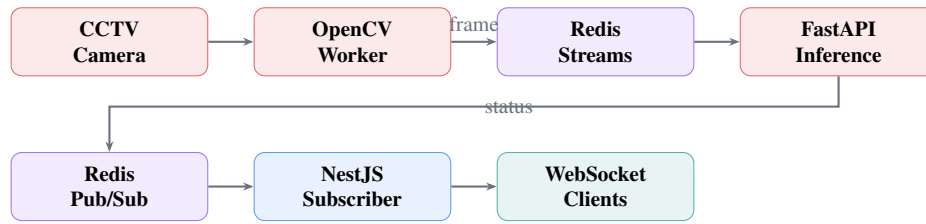


Figure 7: CCTV event pipeline.

6. Security Architecture — Permission-Based RBAC

6.1 Authentication

- JWT **access tokens** with 15-minute expiry.
- JWT **refresh tokens** with 7-day expiry, stored in PostgreSQL with rotation — old token invalidated on every refresh.
- Token payload: `{sub, role, permissions[], iat, exp}`. Permissions are embedded at login so no DB lookup is needed per request.
- All API inputs pass through `class-validator` DTOs before reaching any service layer. No raw `req.body` access in controllers.
- NestJS ↔ FastAPI internal calls use a shared API key (environment variable). FastAPI is not port-exposed to the host — only reachable on the internal Docker network.

6.2 PDPO 2025 Compliance Readiness

Bangladesh’s Personal Data Protection Ordinance (full enforcement ~May 2027) classifies license plate data as personal data because plates link to vehicle owners through BRTA records. Practical readiness steps:

- Detection images and plate logs tagged as personal data in the schema.
- `detection_events` TTL set to 30 days (configurable); no indefinite retention.
- No third-party data sharing endpoints without explicit scope in the API contract.
- Entrance signage requirement tracked as a deployment checklist item for live deployments.
- During development, public datasets (PKLot, CNRPark) are used — no real personal data collected.

6.3 Role Definitions

Role	Description
admin	Platform superuser. Full access to everything. Manages users, approves owners, suspends accounts. Bypasses all permission checks in the guard chain.
owner	CRUD on <i>own</i> spaces, slots, and cameras only. Cannot touch another owner’s data.
authority	Read-only on all spaces and occupancy stats. In future planned: booking info and LPR logs.
driver	Mobile app end user. Search, view, and book slots. No owner or authority data access.

Table 3: CityPulse roles and their scope.

6.4 Permission Catalogue

Permission	Meaning	Granted to
parking:space:create	Create a new parking space	owner, admin
parking:space:read	Read a single space	all roles
parking:space:update	Update own space	owner, admin
parking:space:delete	Delete own space	owner, admin
parking:space:read:all	Read all spaces (not just own)	authority, admin
parking:slot:manage	Manage slots on own spaces	owner, admin
parking:camera:manage	Manage cameras on own spaces	owner, admin
user:read:all	List / read all users	admin
user:update:role	Change a user role	admin
user:suspend	Suspend / activate an account	admin
owner:verify	Approve owner accounts	admin

Table 4: Permissions — spaces, slots, cameras, users.

Permission	Meaning	Granted to
detection:read	Read detection events & occupancy	authority, admin
detection:read:lpr	Read LPR / plate logs (future planned)	authority, admin
jam:read	Read jam alerts (future planned)	authority, admin
booking:create	Create a booking (future planned)	driver, admin
booking:read:own	Read own bookings (future planned)	driver, admin
booking:read:all	Read all bookings (future planned)	authority, admin
booking:cancel	Cancel own booking (future planned)	driver, admin
analytics:read:own	Own space analytics	owner, admin
analytics:read:all	City-wide analytics	authority, admin

Table 5: Permissions — detection, booking, analytics.

6.5 Role to Permission Mapping

Role	Permissions assigned (seeded in DB on init)
admin	All permissions — bypasses guard entirely.
owner	parking:space:create, parking:space:read, parking:space:update, parking:space:delete, parking:slot:manage, parking:camera:manage, analytics:read:own.
authority	parking:space:read:all, detection:read, jam:read, analytics:read:all. Future planned adds: booking:read:all, detection:read:lpr.
driver	parking:space:read, booking:create, booking:read:own, booking:cancel.

Table 6: Role to permission mapping — stored in role_permissions, seeded on init.

6.6 Guard Chain

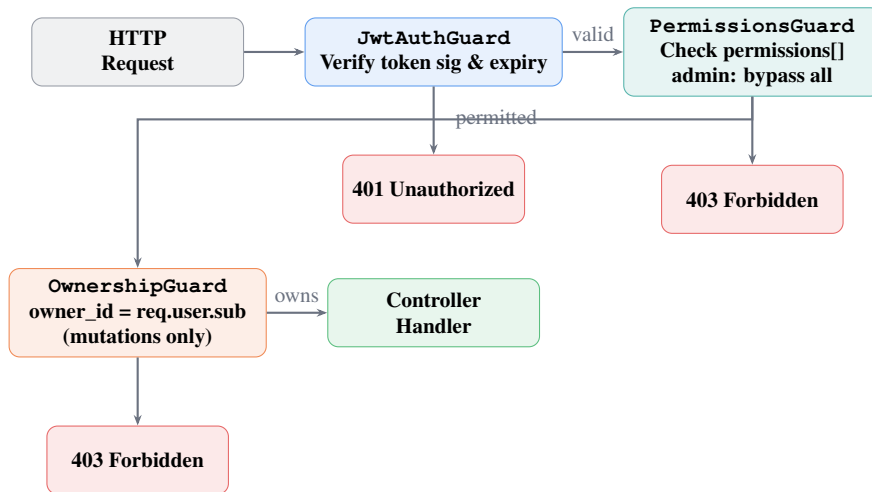


Figure 8: NestJS guard chain — three composable guards.

6.7 JWT Token Payload

```

{
  "sub": "uuid-of-user", "role": "owner",
  "permissions": ["parking:space:create", "parking:space:read",
    "parking:space:update", "parking:space:delete",
    "parking:slot:manage", "parking:camera:manage", "analytics:read:own"],
  "iat": 1751644800, "exp": 1751645700
}

```

6.8 RBAC Entity Relationships

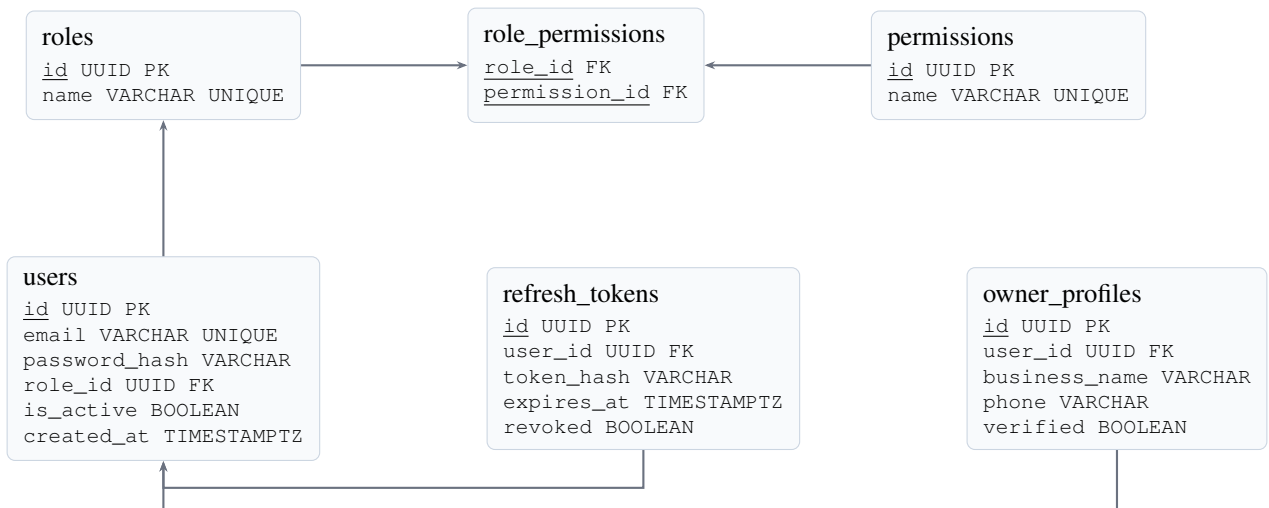


Figure 9: RBAC and user management entity relationships.

7. Database Design

7.1 PostgreSQL Core Schema

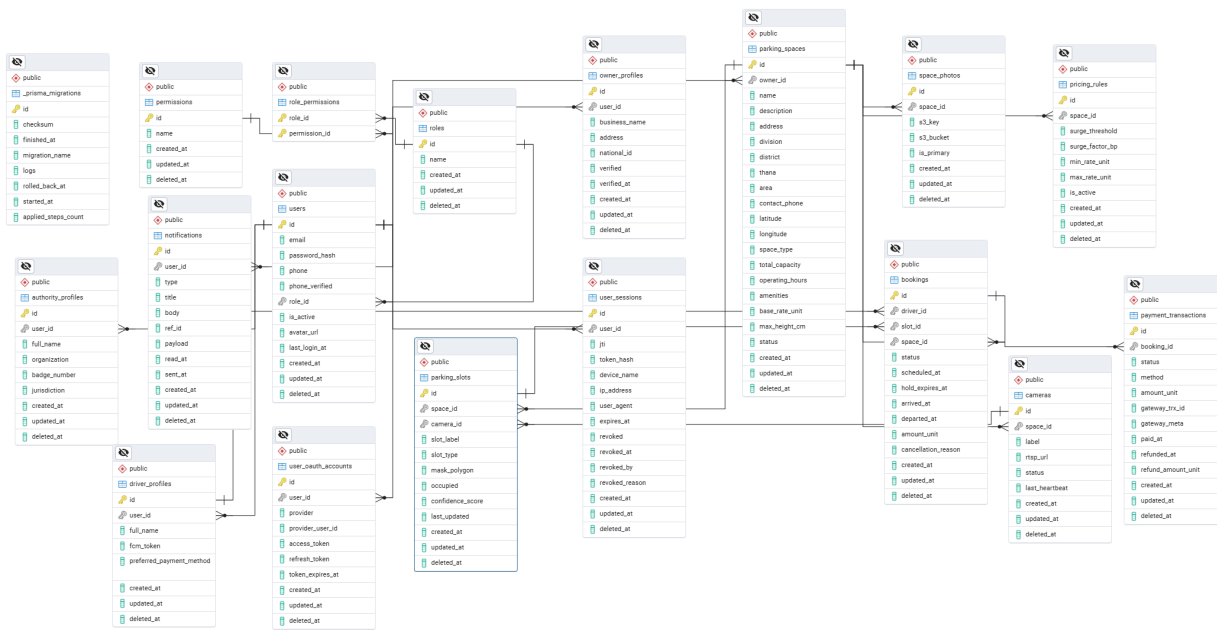


Figure 10: PostgreSQL database schema — full CityPulse ERD.

7.2 MongoDB Collections

```
{ camera_id, lot_id, slot_id, occupied: true, confidence: 0.94,
  frame_timestamp: ISODate, created_at: ISODate }
// Index: { lot_id: 1, frame_timestamp: -1 }
// TTL: { created_at: 1 }, expireAfterSeconds: 2592000
```

```
{ lot_id, bucket: ISODate("2026-07-04T10:20:00Z"),
  total_slots: 24, occupied_count: 18, free_count: 6 }
```

7.3 Redis Key Patterns

```
slot:{slot_id}:state    -> "occupied"|"free"    TTL: 10s
lot:{lot_id}:free_count -> integer           TTL: 10s
slot:{slot_id}:hold     -> booking_id         TTL: 1800s (future planned)
```

7.4 PostGIS Nearby Search Query

INPUT: user GPS location, search radius

1. Find all active parking spaces within radius of user location
2. For each space, count how many slots are currently free
3. Check Redis cache for live free slot count (faster than DB)
4. Sort results by distance (nearest first)
5. Return top 20 results with: name, address, distance, free slot count

OUTPUT: ranked list of nearby parking spaces with live availability

8. Deployment Architecture

Each service is containerized independently and deployed separately. There is no single shared Docker Compose file tying all services together. Services communicate over the network using environment-configured URLs.

8.1 Service Containers

Service	Port	Notes
Next.js Frontend	3000	Served independently, calls NestJS backend via configured API URL
NestJS Backend	3001	Exposes REST API and WebSocket endpoint publicly
FastAPI AI Service	8000	Exposes HTTP endpoint publicly; called by NestJS via configured URL
PostgreSQL	5432	Local: self-hosted container. Online: managed cloud PostgreSQL service. Accessible to NestJS.
MongoDB	27017	Local and online: accessible to NestJS and FastAPI
Redis	6379	Accessible to both NestJS and FastAPI
RabbitMQ	5672	Accessible to NestJS; management UI on 15672
S3 Object Storage	9000	Local: self-hosted container (admin UI on 9001). Online: cloud S3-compatible object storage. Accessible to NestJS.

Table 7: Service deployment — each service runs in its own container with its own port.

8.2 Environment Configuration

Every service reads all configuration from environment variables. No hardcoded credentials or URLs anywhere in the codebase. Each service ships a `.env.example` documenting every required variable. `.env` files are gitignored.

Startup dependency order (when running on the same host): `postgres + mongo + redis + rabbitmq + minio → NestJS → FastAPI → Next.js`.

9. Design Patterns Applied

Pattern	Where	Purpose
Repository Pattern	NestJS modules	Abstracts DB; controllers never query the DB directly
DTO + Validation	All API inputs	<code>class-validator</code> DTOs enforce shape at the boundary
Permission-Based RBAC	AuthModule	Permissions in DB, resolved into JWT; routes guarded by strings
Guard Chain (Composite)	NestJS routes	<code>JwtAuthGuard</code> → <code>PermissionsGuard</code> → <code>OwnershipGuard</code>
Observer / Pub-Sub	Redis Pub/Sub	FastAPI publishes, NestJS subscribes — decoupled
Worker Queue	BullMQ	FCM notifications as background jobs, not blocking handlers
Dead Letter Queue	RabbitMQ	Failed payment events routed for inspection and retry
TTL Optimistic Lock	Redis (future planned)	Slot hold during booking window without a DB lock
Strangler (planned)	Fig NestJS modules	Any module extractable to a separate service in future planned
CQRS (light)	ParkingModule	Read via Redis cache; writes direct to MongoDB

Table 8: Design patterns used in CityPulse.